

Manual del sistema de plugins (DocumentIA)

1) ¿Qué es y qué aporta?

El proyecto `DocumentIA.Plugins` implementa una capa de integración extensible para enriquecer los datos extraídos de documentos según la tipología.

Valor que aporta

- **Extensibilidad por configuración:** permite activar/desactivar integraciones sin cambiar código de la Function principal.
 - **Encadenado por prioridad:** ejecuta múltiples plugins en orden y acumula enriquecimiento de datos.
 - **Soporte multi-protocolo:** REST, SOAP y lógica personalizada en DLL (`custom`).
 - **Resiliencia integrada:** retries + backoff + circuit breaker básico mediante `ResilientPlugin` .
 - **Aislamiento de responsabilidades:** la orquestación vive en `IntegrarActivity` ; cada plugin encapsula su integración.
-

2) Componentes principales

Proyecto base

- `src/backend/DocumentIA.Plugins/Integration/IIntegrationPlugin.cs`
 - Contrato común para cualquier plugin:
 - `InitializeAsync(configuration)`
 - `ExecuteAsync(data)`
 - `HealthCheckAsync()`
 - Define también `IntegrationResult` y `PluginException` .

Gestión y creación

- `PluginConfigLoader`
 - En runtime carga la configuración publicada desde `PluginTipologiaConfigs` en BD.
 - Mantiene modo fichero (`config/tipologias/{tipologiaId}.plugins.json`) solo para pruebas, compatibilidad local o seed historico.
 - Cachea configuración en memoria y permite invalidar/recargar.
- `PluginFactory`
 - Crea instancias según `pluginType` :
 - `rest` → `RestPlugin`
 - `soap` → `SoapPlugin`
 - `custom` → `CustomPlugin`
 - Inicializa el plugin con `configuration` .
 - Si existe `retryPolicy` , envuelve con `ResilientPlugin` .
- `PluginManager`
 - Registra plugins por `pluginKey` .
 - Recupera plugins existentes (evita recreación innecesaria).
 - Ejecuta plugin y normaliza errores a `IntegrationResult` .
 - Expone `HealthCheckAllAsync()` y `ListPlugins()` .

Implementaciones disponibles

- `RestPlugin`
 - Integra HTTP (`GET/POST/PUT/DELETE`).
 - Soporta headers, timeout y auth (`Bearer` , `ApiKey` , `None`).

- Devuelve respuesta parseada en `ResponseData` .
- `SoapPlugin`
 - Construye SOAP Envelope, envía request y parsea campos del `<Body>` .
 - Soporta SOAP 1.1 / 1.2 y Basic Auth opcional.
- `CustomPlugin`
 - Carga una DLL externa con reflexión.
 - Requiere una clase que implemente `ICustomEnricher` .
 - Ejecuta enriquecimiento in-process (sin HTTP).
- `ResilientPlugin`
 - Decorador de resiliencia para cualquier plugin.
 - Retry configurable, backoff exponencial opcional y circuit breaker.

3) Cómo funciona de punta a punta

La ejecución real se produce en `src/backend/DocumentIA.Functions/Activities/IntegrarActivity.cs` :

1. Recibe `IntegrarInput` con tipología y datos extraídos.
2. Carga configuración de plugins de esa tipología (`PluginConfigLoader`).
3. Ordena plugins habilitados por `priority` ascendente.
4. Para cada plugin:
 - Obtiene el plugin del `PluginManager` (o lo crea con `PluginFactory`).
 - Construye payload con:
 - `tipologia`
 - `documentoId`
 - `datosExtraidos` (acumulados)
 - `idActivo` (siempre se envía; puede venir vacío)
 - `metadata`
 - Ejecuta plugin (`ExecutePluginAsync`).
 - Si devuelve `ResponseData` , hace merge sobre `DatosFinales` .
5. Tras ejecutar la cadena, se resuelve `IdActivo` con prioridad:
 - `DatosFinales["idActivo"]` (si algún plugin lo devuelve),
 - `IntegrarInput.IdActivo` (valor original de entrada),
 - `null` si no existe en ninguno de los dos.
6. Si falla un plugin crítico (`priority == 1`), detiene la cadena.
7. Devuelve estado final `OK` , `REVISION` o `ERROR` .

Registro DI en `src/backend/DocumentIA.Functions/Program.cs` :

- `AddHttpClient()`
- `AddSingleton<PluginManager>()`
- `AddSingleton<PluginFactory>()`
- `AddSingleton<PluginConfigLoader>(...)`

4) Configuración de plugins por tipología

Fuente de verdad runtime:

- Tabla `PluginTipologiaConfigs` , gestionada por DocumentIA.Admin o Admin API `/management/plugins-tipologias/{codigo}` .

Ficheros físicos:

- `src/backend/DocumentIA.Functions/config/tipologias/*.plugins.json`

Estos ficheros son seed inicial, plantilla o referencia historica. Pueden estar desactualizados respecto a BBDD y no deben editarse para cambiar produccion ni borrarse sin confirmacion explicita.

Esquema esperado

```
{
  "tipologiaId": "nota.simple.1_4",
  "plugins": [
    {
      "pluginKey": "mock-enrichment",
      "pluginType": "rest",
      "enabled": true,
      "priority": 1,
      "configuration": {
        "baseUrl": "http://localhost:8080",
        "endpoint": "/",
        "authType": "None",
        "timeoutSeconds": 10
      },
      "retryPolicy": {
        "maxRetries": 2,
        "initialDelayMs": 500,
        "exponentialBackoff": true,
        "retryOnStatusCodes": [408, 429, 500, 502, 503, 504]
      }
    }
  ]
}
```

Campos clave

- `pluginKey` : identificador único en el manager.
- `pluginType` : `rest` , `soap` o `custom` .
- `enabled` : activa/desactiva ejecución.
- `priority` : orden de ejecución (menor = antes).
- `returnsIdActivo` (*opcional, anotación documental*): indica que el plugin puede devolver el `idActivo` del activo en su `ResponseData` . No modifica el comportamiento del motor — el pipeline siempre lee `idActivo` directamente de `DatosFinales["idActivo"]` tras la integración.
- `configuration` : parámetros específicos por tipo.
- `retryPolicy` (*opcional*): activa resiliencia.

Configuración por tipo

- **rest**
 - `baseUrl` , `endpoint` , `authType` , `authToken` , `timeoutSeconds` , `headers`
- **soap**
 - `endpoint` , `soapVersion` , `action` , `namespace` , `authType` , `username` , `password` , `timeoutSeconds`
- **custom**
 - `assemblyPath` , `className` , `customConfig`

Nota importante: `PluginFactory` usa el campo `pluginType` (no `type`) y espera valores `rest|soap|custom` .

5) Cómo usarlo (operativa)

Paso 1: registrar/publicar la configuración en BD

Crear o editar la configuracion de plugins desde DocumentIA.Admin o Admin API. Usar `*.plugins.json` solo como plantilla o seed de un entorno nuevo.

Paso 2: si usas plugin custom, compilar y publicar DLL

Hay scripts de apoyo:

- `scripts/compile-all-plugins.ps1`
- `scripts/compile-plugins.ps1`

Ejemplo: `compile-all-plugins.ps1` compila `SarebEnrichments` y copia `SarebEnrichments.dll` a `plugins/`.

Paso 3: ejecutar Functions

Con el host de Functions levantado, `IntegrarActivity` aplicará automáticamente la cadena de plugins definida para la tipología del documento.

Paso 4: observar resultado

El resultado de integración incluye:

- estado global (`OK` , `REVISION` , `ERROR`),
- detalle por plugin (`PluginExecutionResult`),
- `DatosFinales` tras merge acumulado,
- `IdActivoResuelto` (priorizando el `idActivo` devuelto por plugins).

Nota para integraciones GDC

Para facilitar la subida posterior al gestor documental, los plugins deben:

- aceptar `idActivo` en el payload de entrada,
- devolver `idActivo` en `ResponseData` cuando lo obtengan durante el enriquecimiento.

Así, el pipeline conserva un único `IdActivo` resuelto para pasos posteriores (p. ej., actividad de upload a GDC).

6) Crear un plugin nuevo

Opción A: nuevo plugin HTTP/SOAP (dentro de `DocumentIA.Plugins`)

1. Implementar `IIntegrationPlugin` .
2. Añadir caso en `PluginFactory.CreatePluginAsync` .
3. Definir configuración necesaria en `PluginTipologiaConfigs.ConfiguracionJson` mediante Admin API/DocumentIA.Admin.
4. Probar con tests unitarios.

Opción B: plugin custom externo (DLL)

1. Crear proyecto de clase (`net8.0`) que reference `DocumentIA.Plugins` .
2. Implementar `ICustomEnricher` .
3. Compilar DLL y copiar a carpeta `plugins/` .
4. Configurar `pluginType: "custom"` + `assemblyPath` + `className` .

Ejemplo real de implementación:

- `src/enrichments/SarebEnrichments/NotaSimpleEnricher.cs`

7) Buenas prácticas recomendadas

- Mantener `pluginKey` estable y descriptivo.
- Usar `priority = 1` solo para integraciones realmente críticas.
- Definir `retryPolicy` para dependencias externas inestables.
- Evitar lógica de negocio pesada en `IntegrarActivity` ; ubicarla en plugins.

- En plugins custom, controlar excepciones y devolver datos trazables.
 - Versionar cambios de plantillas/seed `*.plugins.json` solo cuando se decida mantener una referencia documental; la configuración viva se versiona en BBDD mediante estados y auditoría.
-

8) Referencias rápidas (código)

- Core plugins: `src/backend/DocumentIA.Plugins/Integration`
- Registro DI: `src/backend/DocumentIA.Functions/Program.cs`
- Ejecución en pipeline: `src/backend/DocumentIA.Functions/Activities/IntegrarActivity.cs`
- Configuración runtime de plugins: tabla `PluginTipologiaConfigs` en BD
- Plantillas/seed historico: `src/backend/DocumentIA.Functions/config/tipologias`
- Ejemplo custom enricher: `src/enrichments/SarebEnrichments/NotaSimpleEnricher.cs`
- Tests unitarios: `src/backend/DocumentIA.Tests.Unit/Plugins`
- Plantillas JSON (incluye local/qa/prod): `docs/contratos/PLANTILLA_PLUGINS_JSON.md`